

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

**UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM - 631 561**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : Y.AJAY SATHWIK
Reg.No : 11249A413
Class : II B.E(CSE)
Subject Code : BCSF2441A12
Subject Name : DATA SCIENCE TOOLKIT LAB

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

**UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM - 631 561**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by

Mr/Ms. Y.Ajay sathwik

**with Reg. No 11249A413 done by of II-B.E.(CSE) in dst
for Lab during the academic year 2025 -2026.**

Station : ENATHUR

Date :

Staf-in-charge

Head of the Department

Submitted for the Practical examination held on _____

Examine-1

Examine -2

INDEX

SI.NO	Program Name	Date	Page .no	Signature
01 (1.1)	Introduction to Python Libraries for Data Science	02/02/2026		
1.2	Pandas	02/02/2026		
<u>1.3</u>	<u>Matplotlib</u>	02/02/2026		
2	Perform Data Exploration and Pre- processing in Python	<u>09/02/2026</u>		
3	Implement Regularized Linear Regression	<u>14/02/2026</u>		
4	Implement Naive Bayes Classifier	23/02/2026		
5	Implement Regularized Logistic Regression	<u>02/03/2026</u>		
6	Build Modules Using Different Ensembling Techniques	09/03/2026		
7	Build Models Using Decision Trees	16/03/2026		
8	Build the Models Using Different Kernel Functions	23/03/2026		

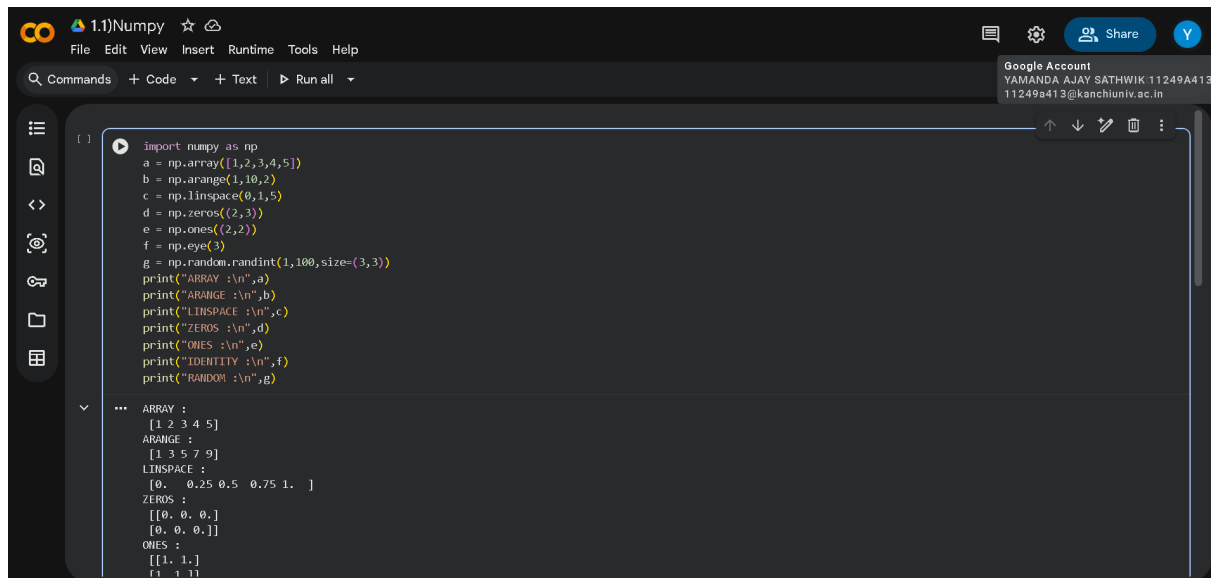
[illegible]

INTRODUCTION TO PYTHON LIBRARIES FOR DATASCIENCE

EXP.NO:-1.1

DATE:-02/02/2026

Aim: To understand the basic Python libraries used in data science, such as NumPy, Pandas, Matplotlib, and Scikit-learn.

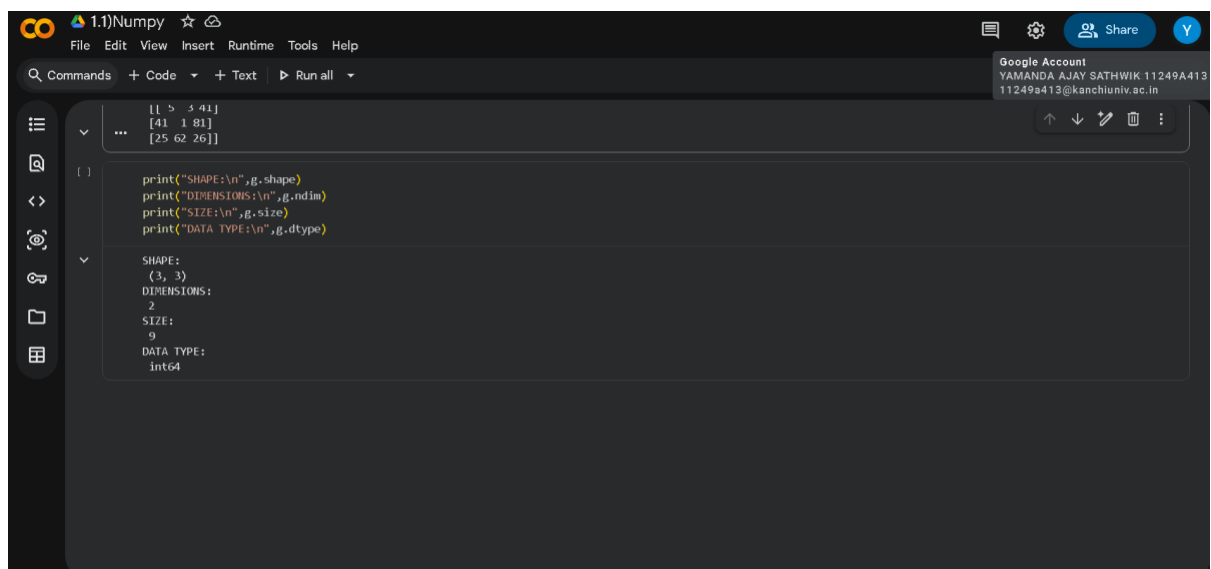


The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the Jupyter logo, the title '1.1)Numpy', and a star icon. Below the top bar is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar with 'Commands' and a 'Run all' button are also present. The main area contains a code cell with the following Python code:

```
import numpy as np
a = np.array([1,2,3,4,5])
b = np.arange(1,10,2)
c = np.linspace(0,1,5)
d = np.zeros((2,3))
e = np.ones((2,2))
f = np.eye(3)
g = np.random.randint(1,100,size=(3,3))
print("ARRAY :\n",a)
print("ARANGE :\n",b)
print("LINESPACE :\n",c)
print("ZEROS :\n",d)
print("ONES :\n",e)
print("IDENTITY :\n",f)
print("RANDOM :\n",g)
```

The output of the code is displayed below the code cell:

```
... ARRAY :
[1 2 3 4 5]
ARANGE :
[1 3 5 7 9]
LINESPACE :
[0. 0.25 0.5 0.75 1. ]
ZEROS :
[[0. 0. 0.]
 [0. 0. 0.]]
ONES :
[[1. 1.]
 [1. 1.]]
RANDOM :
[[ 1  1  1]
 [ 1  1  1]
 [ 1  1  1]]
```



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the Jupyter logo, the title '1.1)Numpy', and a star icon. Below the top bar is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar with 'Commands' and a 'Run all' button are also present. The main area contains a code cell with the following Python code:

```
print("SHAPE:\n",g.shape)
print("DIMENSIONS:\n",g.ndim)
print("SIZE:\n",g.size)
print("DATA TYPE:\n",g.dtype)
```

The output of the code is displayed below the code cell:

```
... SHAPE:
(3, 3)
DIMENSIONS:
2
SIZE:
9
DATA TYPE:
int64
```

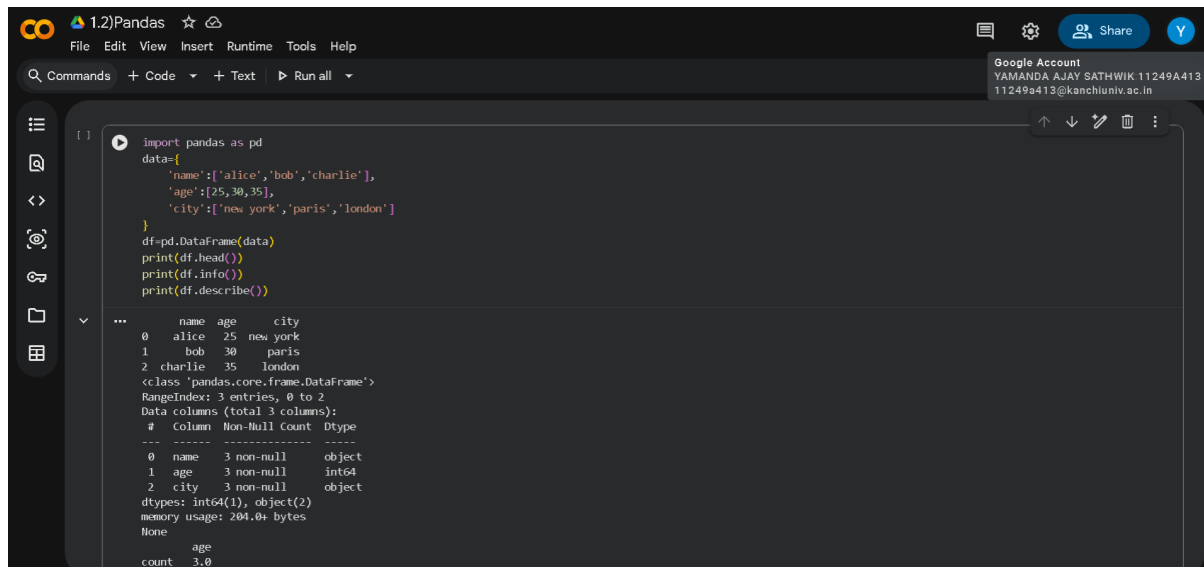
Result: Understood the role of Python libraries in handling, analyzing, and visualizing data. Python libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn are commonly used for these tasks.

EXP.NO:-1.2

DATE:-02/02/2026

Pandas EXP

Aim: To learn how to use Pandas for data loading, manipulation, and analysis.



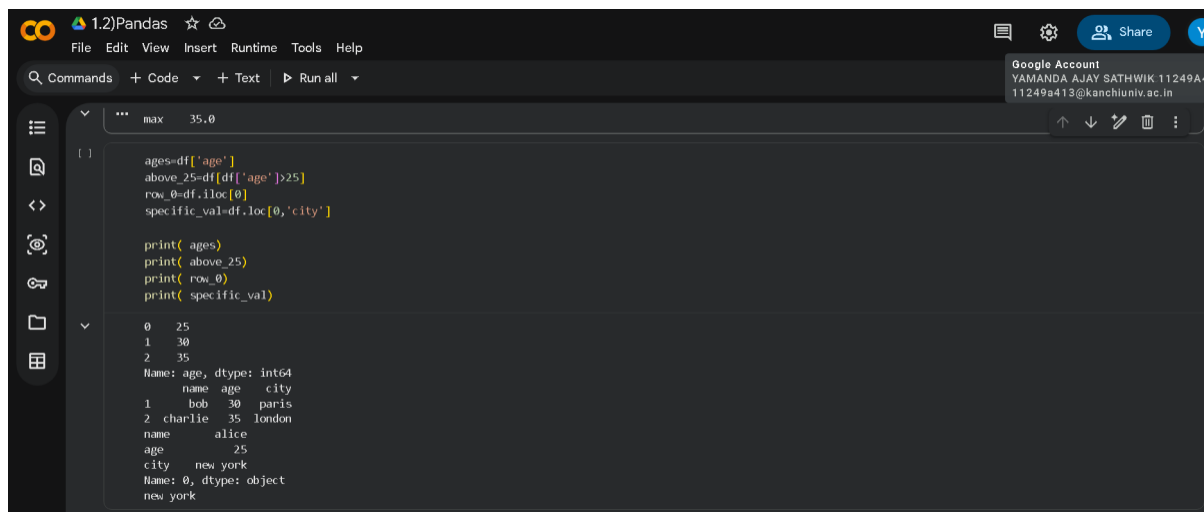
The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the Jupyter logo, the file name '1.2)Pandas', and standard menu items (File, Edit, View, Insert, Runtime, Tools, Help). A search bar and a 'Run all' button are also present. The code cell contains the following Python code:

```
import pandas as pd
data={
    'name':['alice','bob','charlie'],
    'age':[25,30,35],
    'city':['new york','paris','london']
}
df=pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())
```

The output of the code is displayed below the code cell. It shows the first three rows of the DataFrame as a table:

	name	age	city
0	alice	25	new york
1	bob	30	paris
2	charlie	35	london

Below the table, the output of `df.info()` is shown, indicating the DataFrame has 3 entries and 3 columns. The output of `df.describe()` is also displayed, showing the count and dtype for each column.



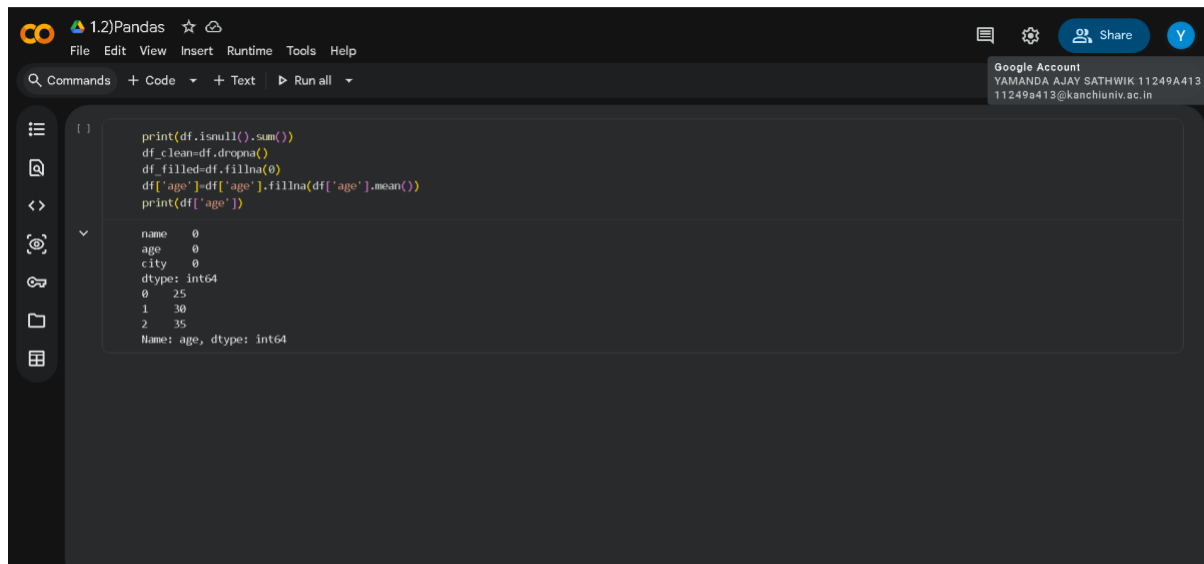
The screenshot shows the same Jupyter Notebook interface. The code cell contains the following Python code:

```
ages=df['age']
above_25=df[df['age']>25]
row_0=df.iloc[0]
specific_val=df.loc[0,'city']

print(ages)
print(above_25)
print(row_0)
print(specific_val)
```

The output of the code is displayed below the code cell. It shows the following results:

```
0    25
1    30
2    35
Name: age, dtype: int64
1    bob    30    paris
2  charlie    35    london
name      alice
age        25
city    new york
Name: 0, dtype: object
new york
```



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the Jupyter logo, the text '1.2)Pandas', and a star icon. Below this is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar contains 'Commands'. To the right of the search bar are buttons for '+ Code', '+ Text', and 'Run all'. In the top right corner, there is a 'Share' button and a user profile icon with the text 'Google Account YAMANDA AJAY SATHWIK 11249A413 11249a413@kanchiuniv.ac.in'.

The notebook has two cells. The first cell contains the following Python code:

```
print(df.isnull().sum())
df_clean=df.dropna()
df_filled=df.fillna(0)
df['age']=df['age'].fillna(df['age'].mean())
print(df['age'])
```

The second cell displays the output of the code, which is a summary of the 'age' column:

```
name      0
age       0
city      0
dtype: int64
0      25
1      30
2      35
Name: age, dtype: int64
```

Result: Successfully performed data manipulation tasks such as creating, modifying, cleaning, filtering, and summarizing datasets. Pandas is widely used for preparing data for analysis and machine learning.

EXP.NO:-1.3

DATE:-02/02/2026

Matplotlib

Aim: To learn how to create graphical visualizations using Matplotlib.

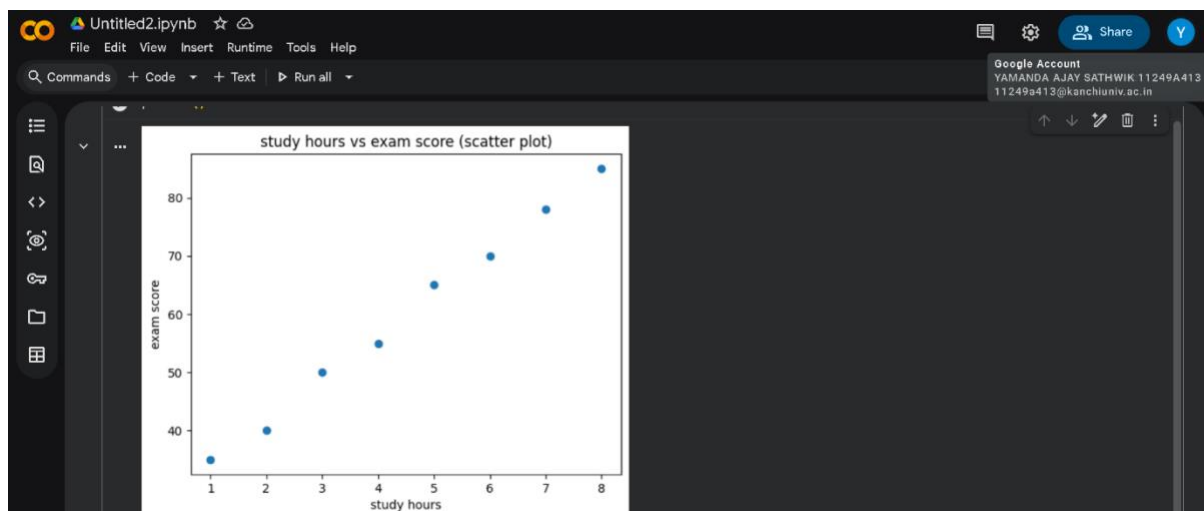
```
Untitled2.ipynb ☆
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all

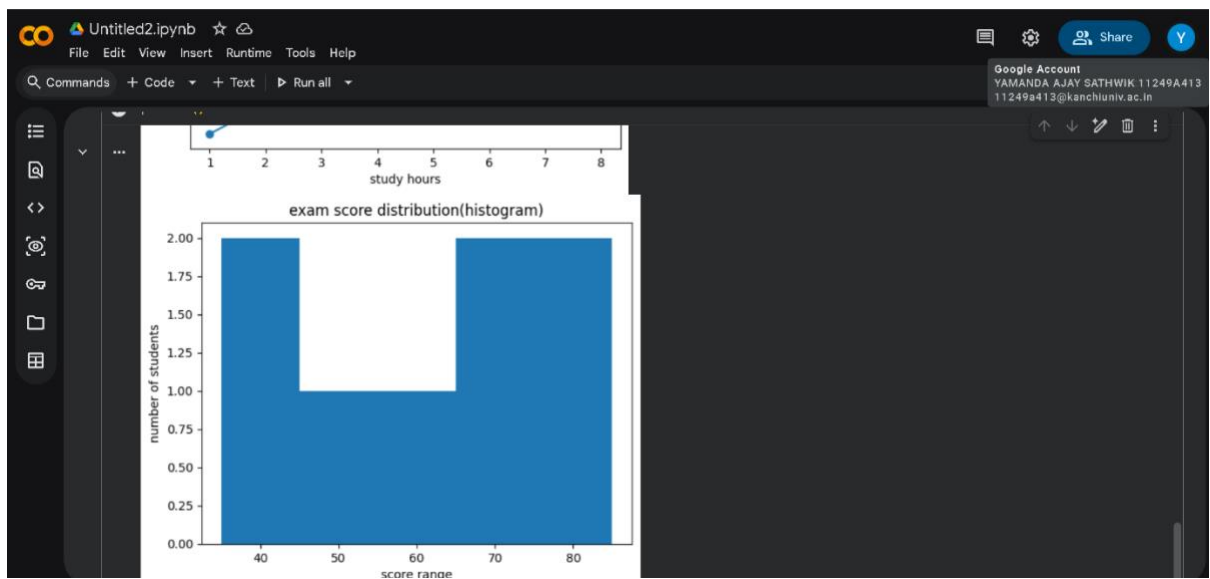
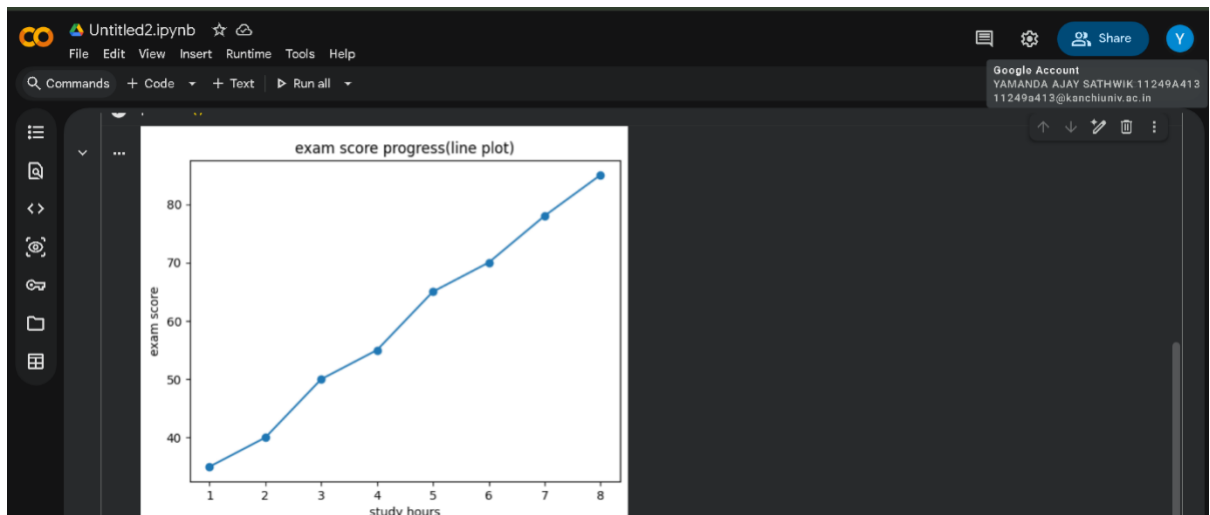
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data={
    'study hours':[1,2,3,4,5,6,7,8],
    'exam score':[35,40,50,55,65,70,85]
}
df=pd.DataFrame(data)

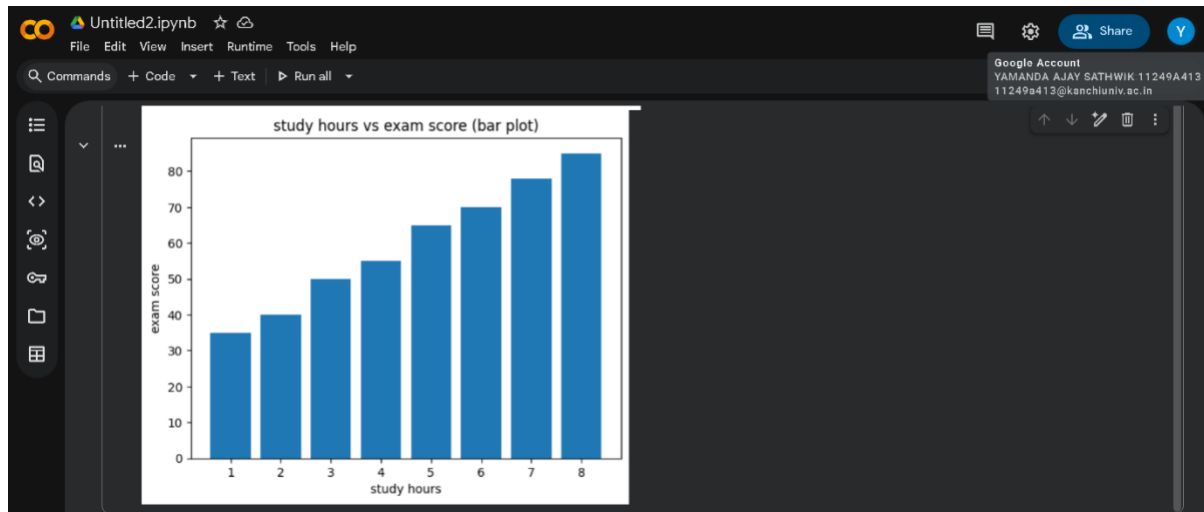
plt.scatter(df['study hours'],df['exam score'])
plt.xlabel('study hours')
plt.ylabel('exam score')
plt.title('study hours vs exam score (scatter plot)')
plt.show()

plt.plot(df['study hours'],df['exam score'],marker='o')
plt.xlabel('study hours')
plt.ylabel('exam score')
plt.title('exam score progress(line plot)')
plt.show()

plt.hist(df['exam score'],bins=5)
plt.xlabel('score range')
plt.ylabel('number of students')
plt.title('exam score distribution(histogram)')
plt.show()
```







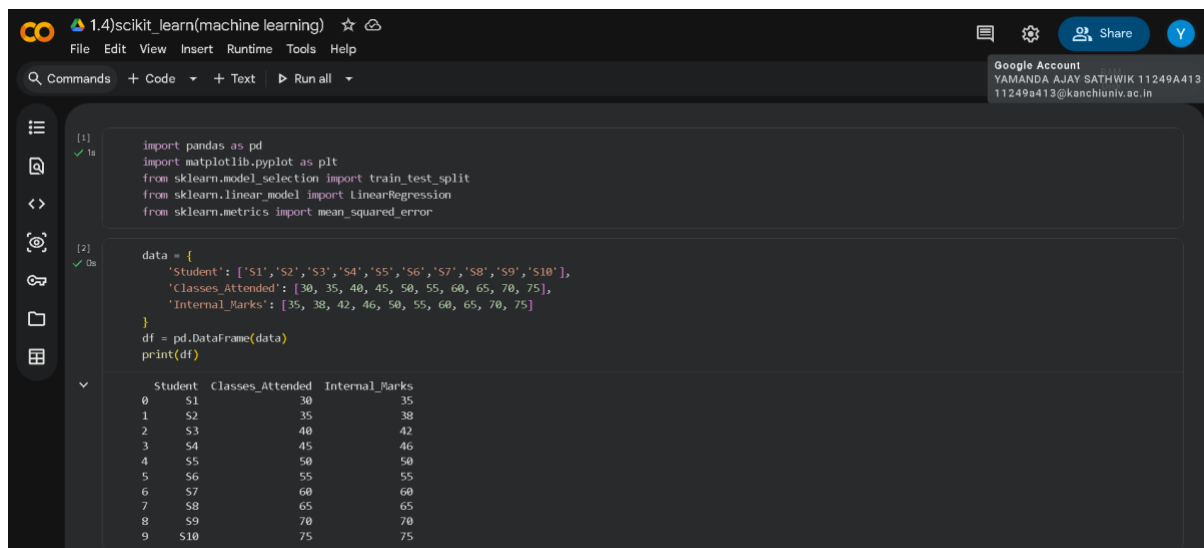
Result: Successfully created plots such as line charts, bar charts, scatter plots, and other basic graphs for data presentation. Matplotlib is commonly used for visualizing x and y points and customizing plots

EXP.NO:-1.4

DATE:-09/02/2026

Scikit-learn

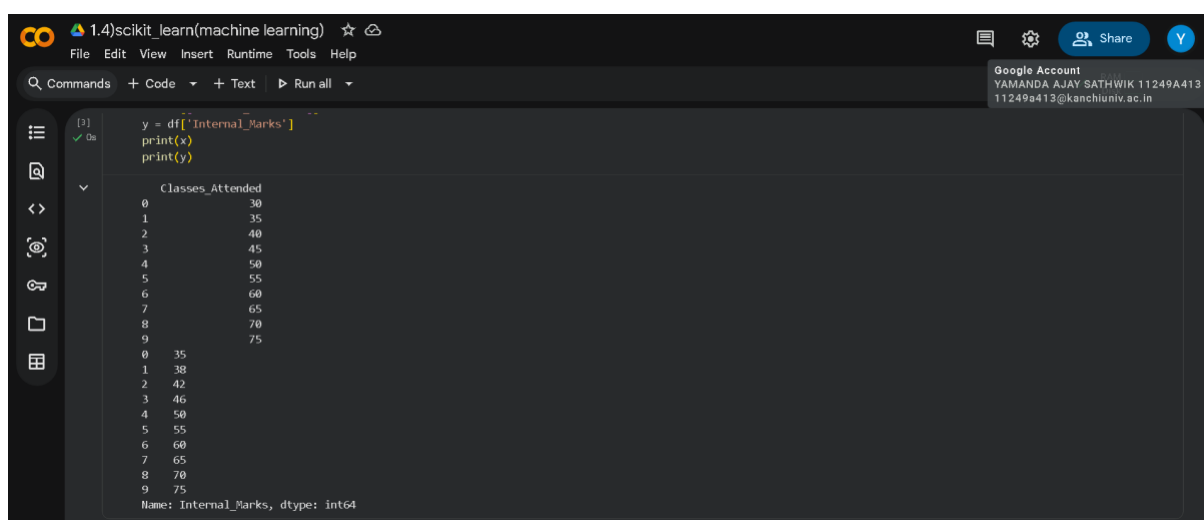
Aim: To understand the use of Scikit-learn for building machine learning models.



```
[1]
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

[2]
data = {
    'Student': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9', 'S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
```

	Student	Classes_Attended	Internal_Marks
0	S1	30	35
1	S2	35	38
2	S3	40	42
3	S4	45	46
4	S5	50	50
5	S6	55	55
6	S7	60	60
7	S8	65	65
8	S9	70	70
9	S10	75	75



```
[3]
y = df['Internal_Marks']
print(x)
print(y)
```

	Classes_Attended
0	30
1	35
2	40
3	45
4	50
5	55
6	60
7	65
8	70
9	75

0 35
1 38
2 42
3 46
4 50
5 55
6 60
7 65
8 70
9 75
Name: Internal_Marks, dtype: int64

```
1.4)scikit_learn(machine learning) ☆ ☁
File Edit View Insert Runtime Tools Help

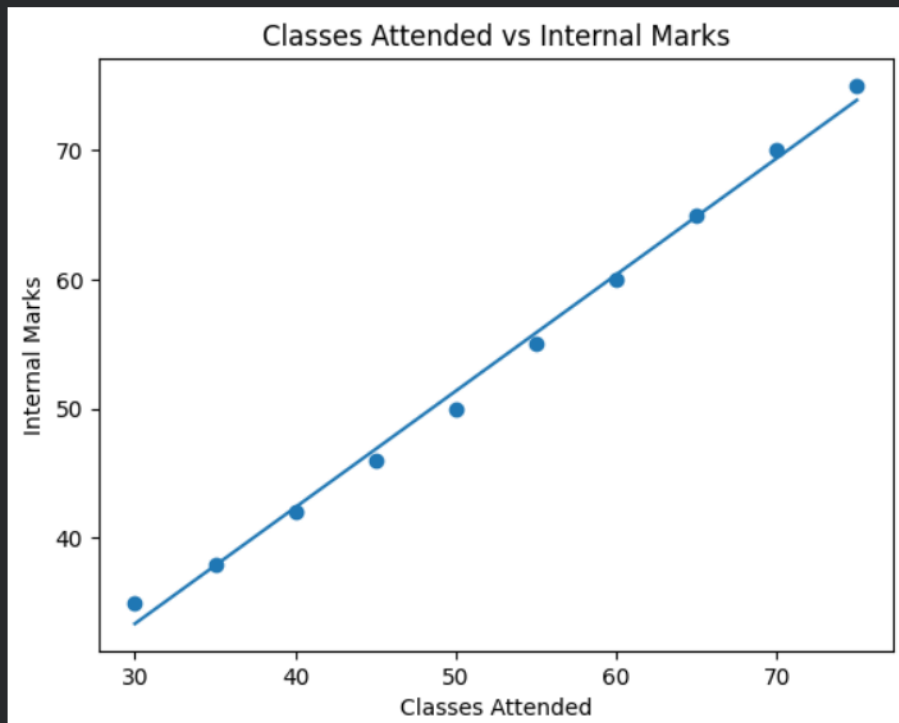
[4] ✓ Os
x = df[['Classes_Attended']]
y = df[['Internal_Marks']]
X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=7
)

[5] ✓ Os
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Define the data and DataFrame (originally in HEC90Lkzz13y)
data = {
    'Student': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9', 'S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)

# Define x and y (originally in bCYu0Rxczrov)
x = df[['Classes_Attended']]
y = df[['Internal_Marks']]

# Perform train_test_split (originally in EqDjEWInz0%-)
X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=7
)
```



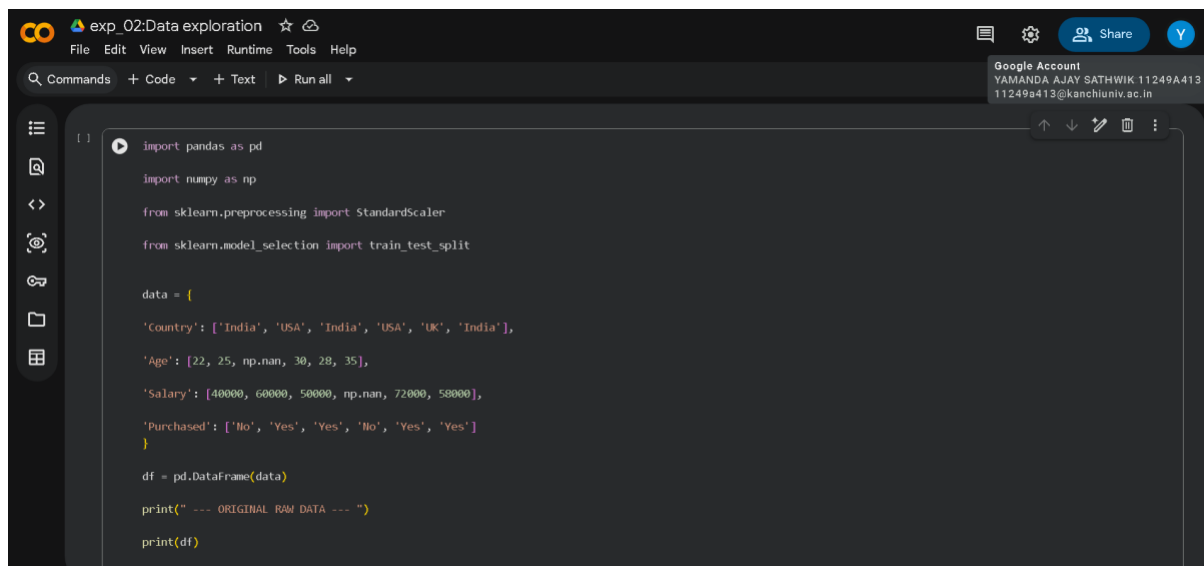
Result: Learned how to apply Scikit-learn for training, testing, and evaluating machine learning models in data science tasks.

EXP.NO:02

DATE:-14/02/2026

Perform Data Exploration and Pre-processing in Python

Aim: To explore datasets and prepare them for analysis by cleaning and transforming the data.



```
import pandas as pd

import numpy as np

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

data = {

    'Country': ['India', 'USA', 'India', 'USA', 'UK', 'India'],

    'Age': [22, 25, np.nan, 30, 28, 35],

    'Salary': [40000, 60000, 50000, np.nan, 72000, 58000],

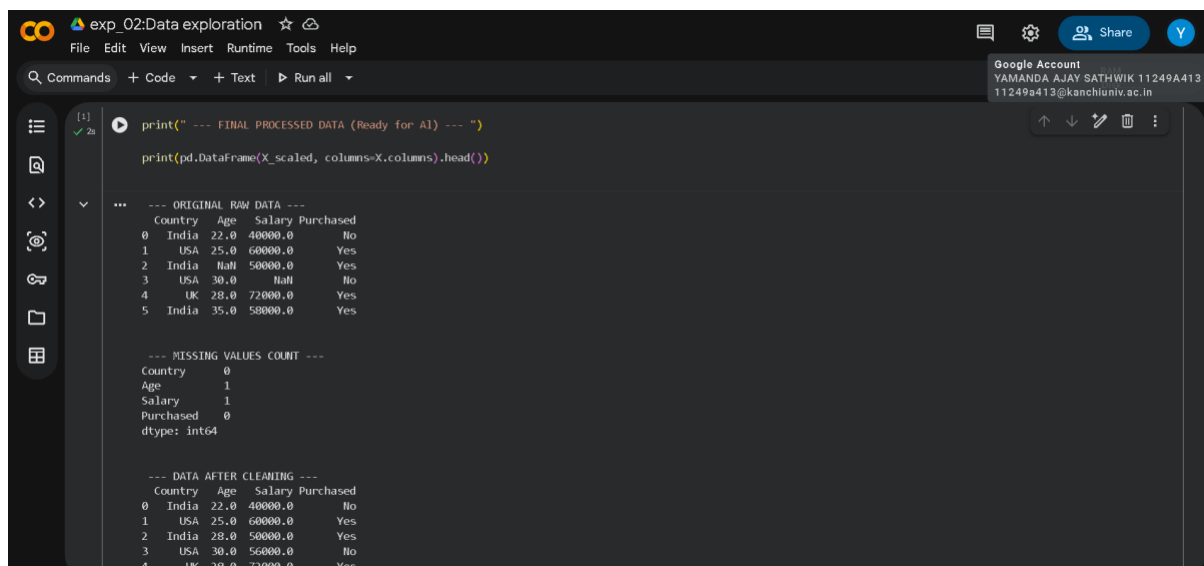
    'Purchased': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes']

}

df = pd.DataFrame(data)

print(" --- ORIGINAL RAW DATA --- ")

print(df)
```



```
print(" --- FINAL PROCESSED DATA (Ready for AI) --- ")

print(pd.DataFrame(X_scaled, columns=X.columns).head())

...

--- ORIGINAL RAW DATA ---
   Country  Age  Salary  Purchased
0   India  22.0  40000.0         No
1    USA  25.0  60000.0         Yes
2   India  NaN  50000.0         Yes
3    USA  30.0   NaN         No
4     UK  28.0  72000.0         Yes
5   India  35.0  58000.0         Yes

--- MISSING VALUES COUNT ---
Country    0
Age        1
Salary     1
Purchased  0
dtype: int64

--- DATA AFTER CLEANING ---
   Country  Age  Salary  Purchased
0   India  22.0  40000.0         No
1    USA  25.0  60000.0         Yes
2   India  28.0  50000.0         Yes
3    USA  30.0  56000.0         No
4     UK  28.0  72000.0         Yes
```

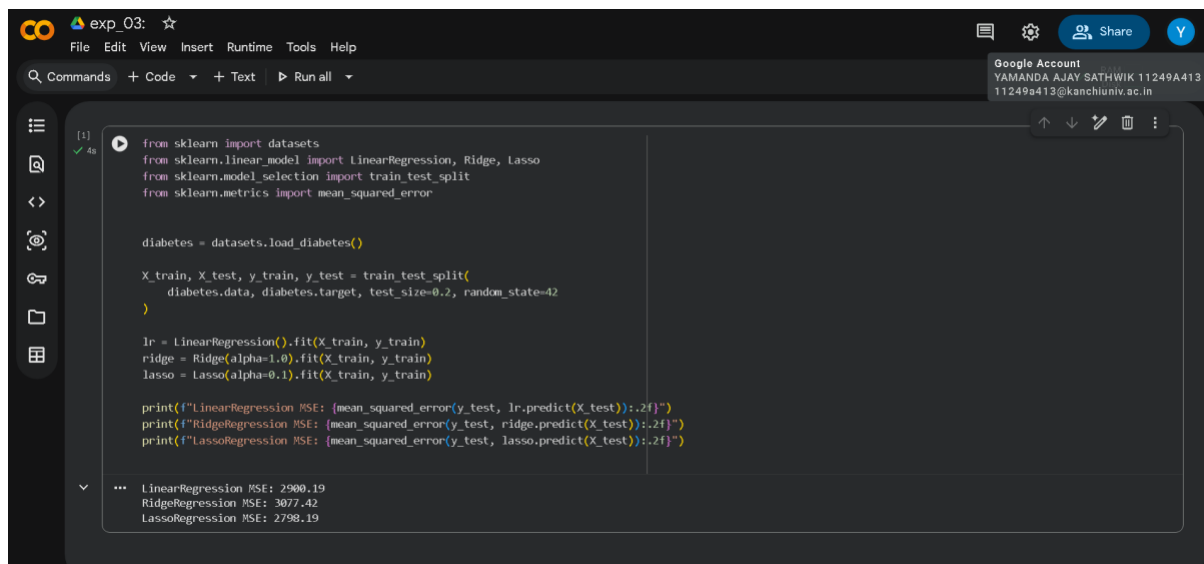
Result: Successfully performed data exploration, handled missing values, encoded categorical features, and scaled data.

EXP.NO:03

DATE:-14/02/2026

Implementation regularized linear regression

AIM:- To implement linear regression with regularization techniques to improve model performance.



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled 'exp_03'. The code in the cell imports necessary libraries from sklearn, loads the diabetes dataset, splits it into training and testing sets, and fits three models: LinearRegression, Ridge, and Lasso. It then prints the Mean Squared Error (MSE) for each model on the test set. The output shows that the Ridge and Lasso models have significantly lower MSE compared to the standard LinearRegression model, indicating better performance due to regularization.

```
[1] ✓ 4s
from sklearn import datasets
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

diabetes = datasets.load_diabetes()

X_train, X_test, y_train, y_test = train_test_split(
    diabetes.data, diabetes.target, test_size=0.2, random_state=42
)

lr = LinearRegression().fit(X_train, y_train)
ridge = Ridge(alpha=1.0).fit(X_train, y_train)
lasso = Lasso(alpha=0.1).fit(X_train, y_train)

print(f"LinearRegression MSE: {mean_squared_error(y_test, lr.predict(X_test)):.2f}")
print(f"RidgeRegression MSE: {mean_squared_error(y_test, ridge.predict(X_test)):.2f}")
print(f"LassoRegression MSE: {mean_squared_error(y_test, lasso.predict(X_test)):.2f}")

...
LinearRegression MSE: 2900.19
RidgeRegression MSE: 3077.42
LassoRegression MSE: 2798.19
```

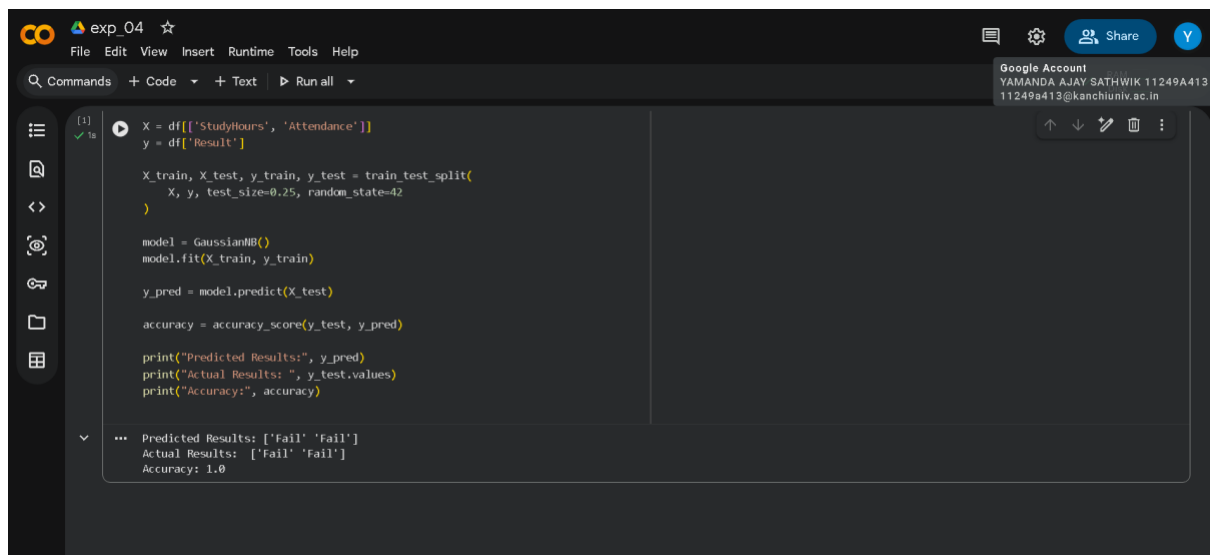
Result: Built a regularized linear regression model and observed reduced overfitting

EXP.NO:04

DATE:-23/02/2026

Implementation naive bayes classified

AIM:- To implement a Naive Bayes classification model for prediction tasks



The screenshot displays a Jupyter Notebook interface with a dark theme. The notebook is titled 'exp_04'. The code cell contains the following Python code:

```
X = df[['StudyHours', 'Attendance']]
y = df['Result']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Predicted Results:", y_pred)
print("Actual Results: ", y_test.values)
print("Accuracy:", accuracy)
```

The output cell shows the following results:

```
... Predicted Results: ['Fail' 'Fail']
Actual Results:  ['Fail' 'Fail']
Accuracy: 1.0
```

The interface includes a sidebar with icons for file explorer, search, and other notebook functions. The top bar shows the file name 'exp_04' and a star icon. The bottom bar displays the Google Account 'YAMANDA AJAY SATHWIK 11249A413' and the email '11249a413@kanchiuniv.ac.in'.

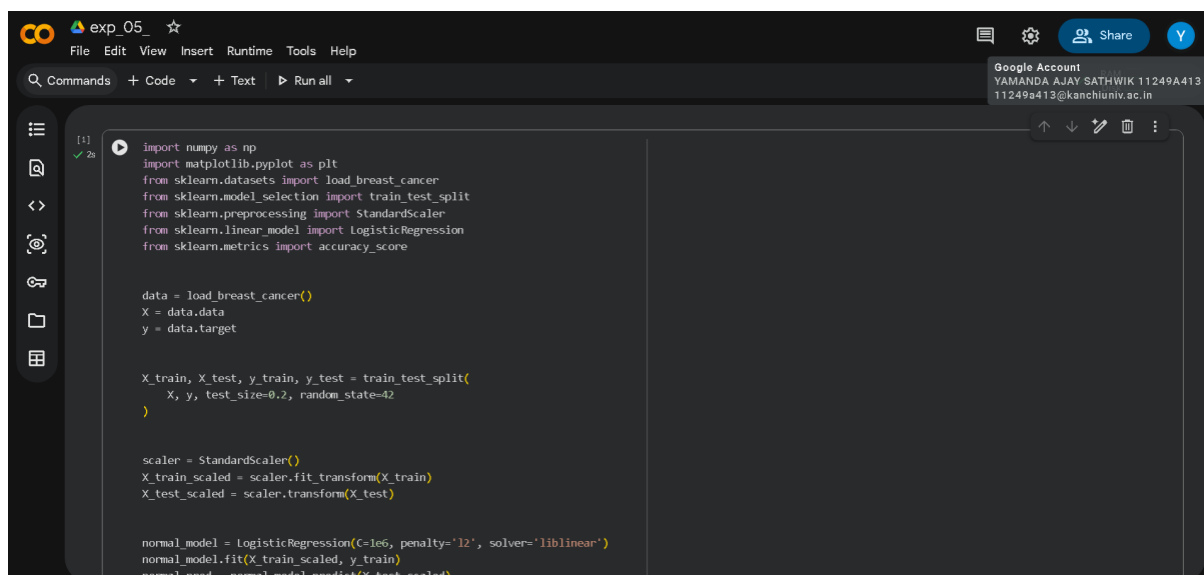
Result: Successfully trained and tested a Naive Bayes classifier and obtained classification results.

EXP.NO:05

DATE:- 2/03/2026

Logistic regression

AIM: To implement logistic regression with regularization for classification.



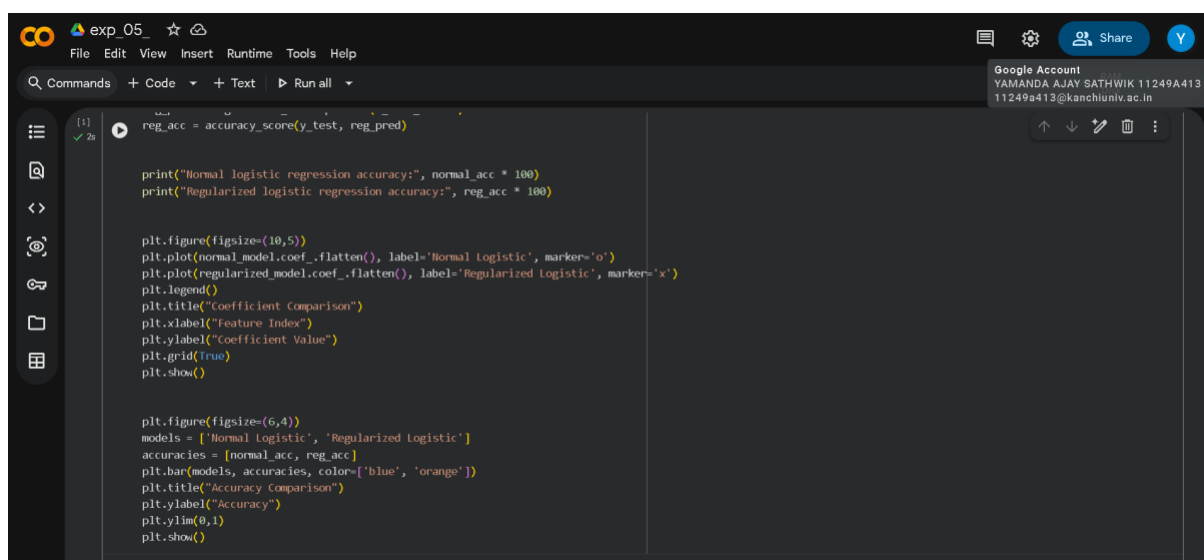
```
[1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)
normal_acc = normal_model.score(X_test_scaled, y_test)
```

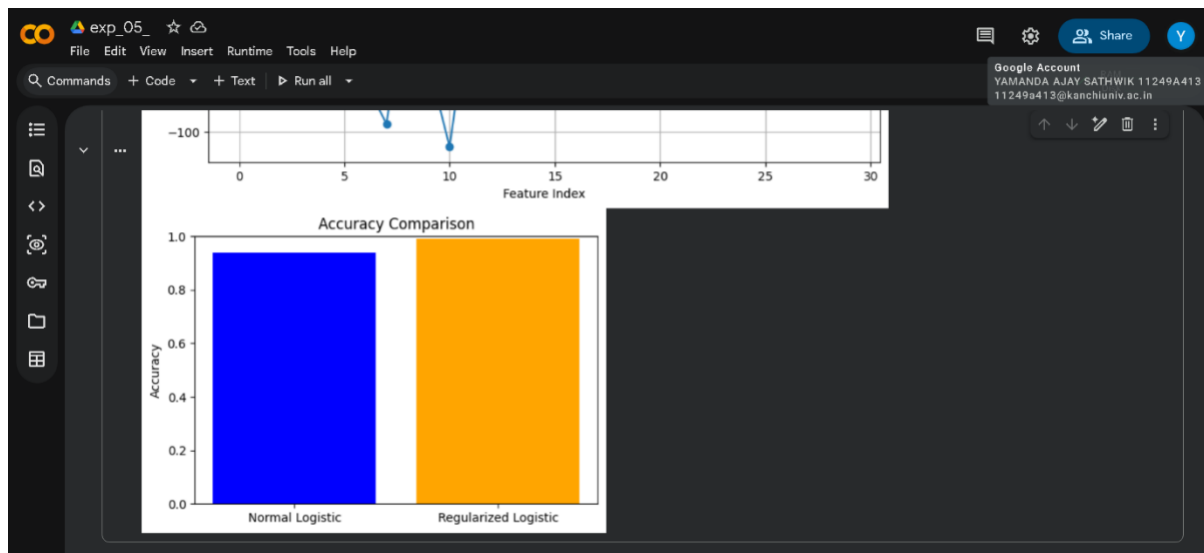


```
[1]: reg_acc = accuracy_score(y_test, reg_pred)

print("Normal logistic regression accuracy:", normal_acc * 100)
print("Regularized logistic regression accuracy:", reg_acc * 100)

plt.figure(figsize=(10,5))
plt.plot(normal_model.coef_.flatten(), label='Normal Logistic', marker='o')
plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic', marker='x')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.grid(True)
plt.show()

plt.figure(figsize=(6,4))
models = ['Normal Logistic', 'Regularized Logistic']
accuracies = [normal_acc, reg_acc]
plt.bar(models, accuracies, color=['blue', 'orange'])
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.ylim(0,1)
plt.show()
```



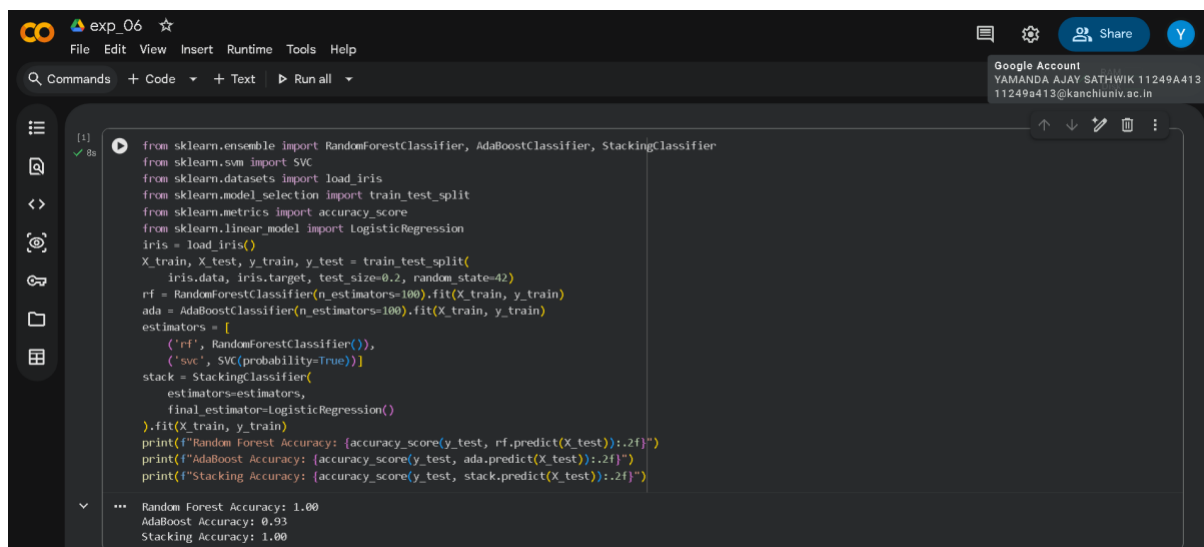
Result: Built a regularized logistic regression model and evaluated its classification accuracy.

EXP.NO:06

DATE:-09/03/2026

Build models using different ensemble techniques

AIM:- To understand and implement ensemble learning methods.



The screenshot shows a Jupyter Notebook window titled 'exp_06'. The code in the cell imports necessary libraries from sklearn, loads the iris dataset, splits it into training and testing sets, and then builds three ensemble models: RandomForestClassifier, AdaBoostClassifier, and StackingClassifier. The output shows the accuracy of each model on the test set.

```
[1]: from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
).fit(X_train, y_train)
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

*** Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Result: Successfully built models using ensemble techniques such as bagging, boosting, or random forest

EXP.NO:07

DATE:-16/03/2026

Build models using decision trees

Aim: To construct decision tree models for classification or regression.



The screenshot displays a Jupyter Notebook environment with a dark theme. The notebook is titled 'exp_07_'. The code cell contains the following Python code:

```
[1]: from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
dt_clf = DecisionTreeClassifier(max_depth=3)
dt_clf.fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

The output cell shows the following results:

```
... Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.          0.          0.39426873 0.60573127]
```

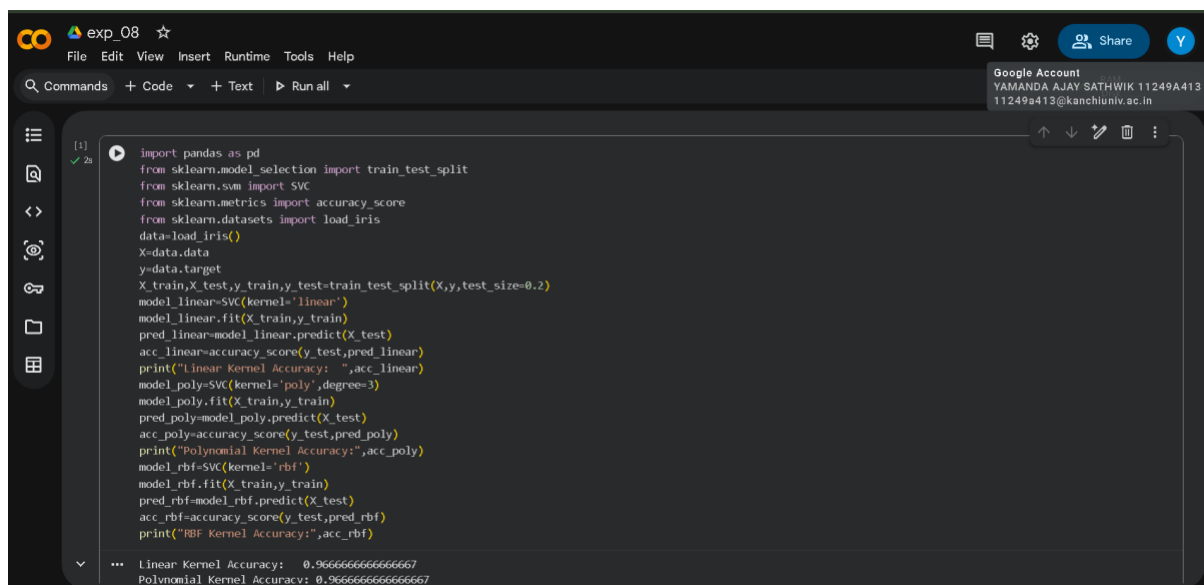
Result: Successfully trained a decision tree model and used it for prediction

EXP.NO:08

DATE:-23/03/2026

support vector machine (svm) with diffrent kernels

Aim: To study and implement kernel-based machine learning models.



```
[1] import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data=load_iris()
X=data.data
y=data.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
model_linear=SVC(kernel='linear')
model_linear.fit(X_train,y_train)
pred_linear=model_linear.predict(X_test)
acc_linear=accuracy_score(y_test,pred_linear)
print("Linear Kernel Accuracy: ",acc_linear)
model_poly=SVC(kernel='poly',degree=3)
model_poly.fit(X_train,y_train)
pred_poly=model_poly.predict(X_test)
acc_poly=accuracy_score(y_test,pred_poly)
print("Polynomial Kernel Accuracy:",acc_poly)
model_rbf=SVC(kernel='rbf')
model_rbf.fit(X_train,y_train)
pred_rbf=model_rbf.predict(X_test)
acc_rbf=accuracy_score(y_test,pred_rbf)
print("RBF Kernel Accuracy:",acc_rbf)

... Linear Kernel Accuracy:  0.9666666666666667
Polynomial Kernel Accuracy: 0.9666666666666667
```

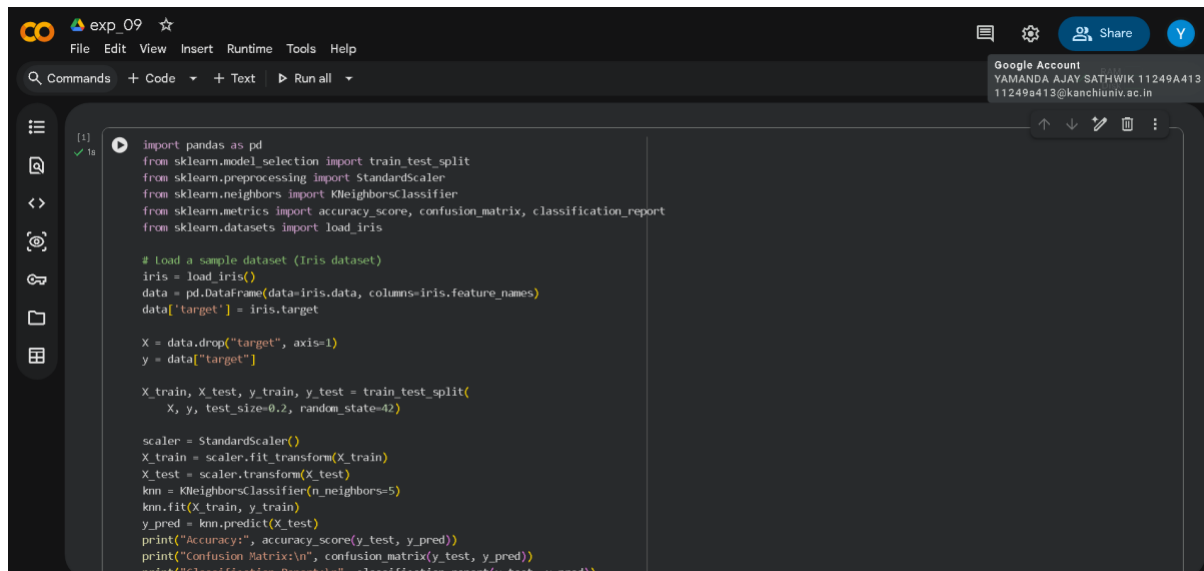
Result: Built models using different kernel functions and compared their performance.

EXP.NO:09

DATE:- 23/03/2026

Implement K-Nearest Neighbours (KNN) Classification

Aim: To implement the KNN algorithm for classification tasks.



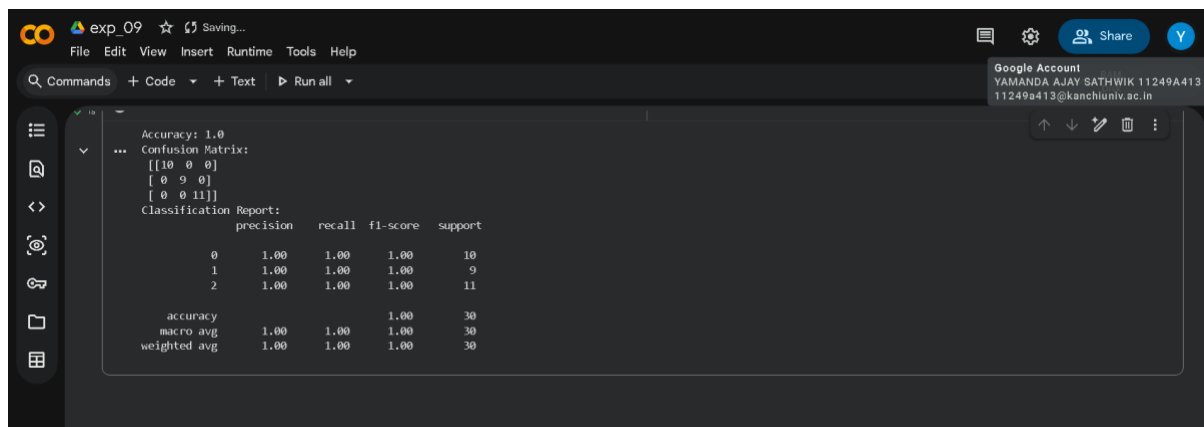
```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.datasets import load_iris

# load a sample dataset (Iris dataset)
iris = load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)
data['target'] = iris.target

X = data.drop("target", axis=1)
y = data["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```



```
Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Result: Successfully classified data using the KNN algorithm and evaluated the model

EXP.NO:10

DATE:- 23/03/2026

K-Mean Clustering with PCA and Elbow Method

Aim: To perform clustering using K-means and determine the optimal number of clusters using PCA and the elbow method.

```
exp_10
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all
Google Account
YAMANDA AJAY SATHWIK 11249A413
11249a413@kanchiuniv.ac.in

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import os
from google.colab import files # Import files module

file_name = "mall customer.csv"

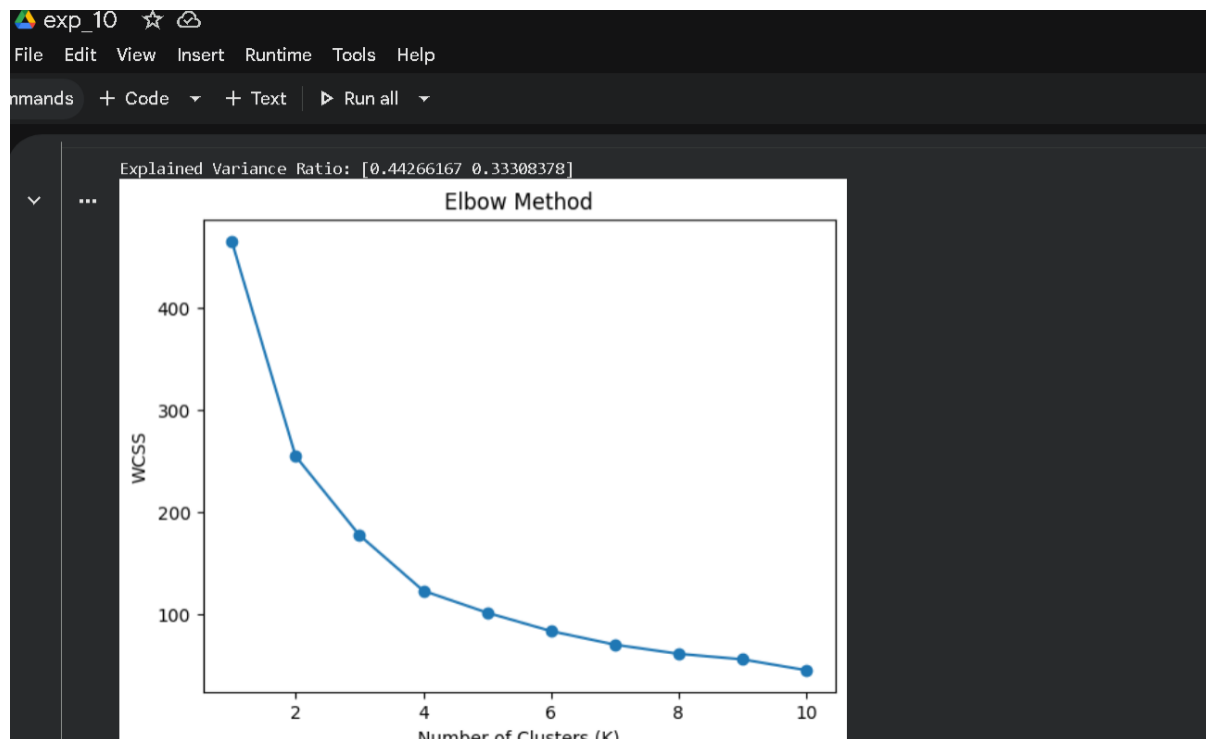
# Check if the file exists, if not, prompt user to upload
if not os.path.exists(file_name):
    print(f"File '{file_name}' not found. Please upload the file.")
    uploaded = files.upload()
    if file_name not in uploaded:
        print(f"Warning: '{file_name}' was not uploaded. Please ensure you upload the correct file.")
        # Optionally, you might want to exit or raise an error here if the file is critical.
        raise FileNotFoundError(f"Required file '{file_name}' not found after upload attempt.")
    else:
        print(f"'{file_name}' uploaded successfully. Continuing with execution.")

data = pd.read_csv(file_name)
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
```

```
exp_10
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all
Google Account
YAMANDA AJAY SATHWIK 11249A413
11249a413@kanchiuniv.ac.in

raise FileNotFoundError(f"Required file '{file_name}' not found after upload attempt.")
else:
    print(f"'{file_name}' uploaded successfully. Continuing with execution.")

data = pd.read_csv(file_name)
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.plot(range(1,11), wcss, markers='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0)
clusters = kmeans.fit_predict(X_pca)
plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters)
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

Result: Successfully applied K-means clustering, reduced dimensions using PCA, and identified the best cluster count using the elbow method.

